

La généricité & la gestion des exceptions

Objectif :

L'objectif de ce TP est de maîtriser les concepts de la généricité et gestion des exceptions en java.

Compte rendu :

- Ce TP est un travail individuel, à rendre avant le Lundi jj/mm/2026.
- Comprimez vos fichiers (codes Java et compte rendu) dans un fichier **Nom-Prénom.zip**.
- Envoyez ce fichier compressé par **e-mail à l'adresse** : « mcherradi.ensah@gmail.com ». Veuillez spécifier "TP5-JSE" dans l'objet du mail.

PARTIE 1 — Exceptions système (surveillées & non surveillées)

Exercice 1 (Division sécurisée) :

Écrire un programme Java qui permet de calculer la division de deux entiers saisis par l'utilisateur.

Contraintes

1. Utiliser un bloc `try/catch`
2. Gérer :
 - division par zéro
 - saisie invalide
3. Utiliser aussi `throw` pour vérifier si le diviseur est égal à zéro

Exercice 2 (NullPointerException) :

Créer une méthode qui affiche la longueur d'une chaîne.

Contraintes

- Tester le cas où la chaîne est `null`
- Éviter l'exception sans utiliser `try/catch`
- Puis proposer une version avec gestion via `try/catch`

Exercice 3 (Tableau sécurisé) :

Créer un tableau de taille 5 et permettre à l'utilisateur d'accéder à un index.

Contraintes

- Gérer `ArrayIndexOutOfBoundsException`
- Proposer :

- une version avec `if`
- une version avec `try/catch`

Exercice 4 (*Conversion d'entier*) :

Convertir une chaîne en entier.

Contraintes

- Gérer `NumberFormatException`
- Afficher un message clair

Exercice 5 (*Choix d'exception*) :

Créer une méthode : `int racineCarree(int x)`

Contraintes

- si $x < 0 \rightarrow$ lever une exception
- choisir entre (avec justification) :
 - `IllegalArgumentException`
 - autre exception

Exercice 6 (*IllegalStateException*) :

Créer une classe `Machine` avec : état : ON / OFF

Méthode : `void demarrer()`

Contraintes

- si déjà ON $\rightarrow$ exception

Exercice 7 (*Propagation*) :

Créer 3 méthodes : $A \rightarrow B \rightarrow C$

Contraintes

- C lance une exception
- B ne la gère pas
- A la gère

Exercice 8 (*Checked Exception*) :

Créer une méthode : `void verifierAge(int age) throws Exception`

Contraintes

- si `age < 18` → exception
- gérer dans `main`

Exercice 10 (Comparaison) :

Refaire exercice 9 avec :

- `Exception`
- `RuntimeException`

Comparer les deux approches

PARTIE 2 — Exceptions personnalisées (checked & unchecked)

Exercice 1 (Compte bancaire) :

Créer une classe `CompteBancaire` (`code`, `solde`) avec les méthodes :

```
void retirer(double montant) ; void verser(double montant) ;
```

Contraintes

- Créer une exception : `SoldeInsuffisantException`
- lever exception si `montant > solde`
- gérer dans `main`

Exercice 2 (Montant invalide) :

Créer une exception: `MontantInvalideException` ; dans le cas ou : `montant ≤ 0` → exception

Exercice 3 (Double validation) :

Modifier `retirer()` pour gérer :

- `montant invalide`
- `solde insuffisant`

Exercice 4 (Inscription Utilisateur) :

Créer une méthode : `void inscrire(String email, int age)`

Contraintes

- `email invalide` → `EmailInvalideException`

- âge < 18 → AgeInvalideException

Créer les deux exceptions

Exercice 5 (Authentication) :

Créer une méthode : `void login(String username, String password)`

Contraintes

- mauvais login → AuthenticationException
- message : "Identifiants incorrects"

Exercice 6 (Gestion de stock) :

Créer une classe `Produit` avec : `void retirerDuStock(int quantite)`

Contraintes

- quantité > stock → StockInsuffisantException

Exercice 7 (Téléchargement) :

Créer : `void telechargerFichier(double taille)`

Contraintes

- taille > limite → QuotaDepasseException

Exercice 8 (Validation de formulaire) :

Créer : `void validerFormulaire(String nom, String email)`

Contraintes

- champ vide → ChampObligatoireException

Exercice 9 (Paiement) :

Créer : `void payer(double montant)`

Contraintes

- montant > plafond → PaiementRefuseException
- carte expirée → CarteExpireeException

Exercice 10 (*Conception*) :

- Quand utiliser checked ?
- Quand utiliser unchecked ?
- Pourquoi créer une exception personnalisée ?

PARTIE 3 — Généricité

Exercice 1 :

Créer une classe `Boite<T>` :

- attribut : `contenu`
- méthodes : `setContenu`, `getContenu`

Tester avec :

- `String`
- `Integer`

Exercice 2 :

Créer `Paire<T, U>` :

- attributs : `premier`, `second`
- méthode `afficherPaire()`

Exercice 3 :

Créer :

```
interface Calcul<T> {  
    T addition(T a, T b);  
}
```

Implémenter pour :

- `Integer`
- `Double`

Exercice 4 :

Créer :

```
interface Compareur<T> {  
    int comparer(T a, T b);  
}
```

Implémenter pour :

- Integer
- String (longueur)

Exercice 5 :

Créer :

```
interface Compareur<T> {  
    int comparer(T a, T b);  
}
```

Implémenter pour :

- Integer
- String (longueur)

Exercice 6 :

Créer :

- `public static <T> void afficherTableau(T[] tableau)`
- `public static <T> T getPremier(T[] tableau)`

Exercice 7 :

Créer : `public static <T extends Number> double somme(T a, T b)`

Exercice 8 :

Créer :

```
class Animal<T> {  
    T nom;  
}
```

Puis : `class Chien extends Animal<String>`

Exercice 9 :

Créer :

```
class Vehicule<T> {  
    T vitesse;  
}
```

Puis :

```
class Voiture<T> extends Vehicule<T>
```

Exercice 10 :

Créer :

```
class Repository<T> {  
    void save(T obj) {}  
}
```

Puis : `class UserRepository extends Repository<User>`

Exercice 11 :

Créer : `void afficherListe(List<?> liste)`

Exercice 12 :

Créer : `void afficherNombres(List<? extends Number> liste)`

Tester avec :

- `List<Integer>`
- `List<Double>`